

Fundamentals of Programming

COMP5005 Assignment

Robotic Warehouse

Semester 1, 2026 v1.0

Discipline of AI and Data Science
Curtin University

1. Preamble

In practicals you have implemented and learned about simulations, object-orientation and (soon) how to automate the running of multiple simulations. In this assignment, you will be making use of this knowledge to extend a given simulation to provide more functionality, complexity and allow automation. You will then report on your design and implementation, and the results generated by the simulation.

2. The Challenge

You will be simulating a robotic warehouse in which autonomous robots collect goods from shelf locations and return them to their home corners in a grid-based environment.

The warehouse should be represented as a grid world. Robots are initialised only at the four corners of the warehouse. Shelves act as obstacles and are arranged in a structured layout, similar to library shelves or supermarket aisles, with walking gaps between shelf columns. Goods are stored at shelf addresses using grid coordinates (x, y).

Each robot should identify the nearest available good, travel to its location, collect one item, and then return to its initial corner. Robots may overlap with one another, but they must not move through shelf locations. If a target good becomes unavailable before a robot reaches it, the robot should update its target and continue operating.

The number of shelf locations is fixed by the warehouse layout, but the number of robots and the number of goods should be variable. Robots may be added or removed manually through the simulation configuration. Goods should be generated automatically, either when the simulation begins or through additional logic that you design. Multiple goods may exist at the same shelf location.

We will provide some sample code to start this assignment, and additional code showing a range of possible approaches. For this assignment, you will develop code to model robots, goods, and warehouse terrain features. Your task is to extend the code and then showcase your simulation by varying input parameters to show how they affect the overall behaviour of the warehouse system.

Note: You do not have to use the supplied sample code. However, any other code that you have not written yourself (e.g. sourced from others, online, or generated) will not receive marks. Lecture, practical, and test materials from COMP1005/5005 are exempt from this rule, but they must still be referenced appropriately.

Remember : Think before you code!

You can do a lot of the assignment planning on paper before any coding. The Feature column of the **Traceability Matrix** should be filled in before coding, then used for planning the coding project and as a checklist as you work through the assignment.

The assessable features for **warehouse.py** include:

1. **Rides:** Robots should be represented as objects that “know” their position, home corner, state, and target good. Robots should be able to move through the warehouse, collect one good at a time, and return it to their home corner.
Prompts: How will you represent robot state? What robot states will be needed, e.g. idle, moving to good, returning? How will the robot decide which good to target? How will the robot behave if its target is no longer available?
2. **Goods / Targeting:** Goods should be represented in a way that supports their location and availability. A location may contain more than one good. Robots should move toward the nearest available good and collect one item at a time.
Prompts: How will you represent multiple goods at one shelf location? How will you decide whether a good is available? How will the system update when one robot takes an item before another robot arrives?
3. **Task Allocation / Pickup-Return Workflow:** The simulation should keep track of each robot’s task progression, including targeting, pickup, carrying, and return. This is the workflow logic of the warehouse system.
Prompts: Which Python data structures are appropriate for representing task allocation and item quantities? Will robots reserve goods in advance, or only check availability on arrival? How will delivery completion be represented?
4. **Warehouse Terrain:** The warehouse map should contain shelves and walkable aisles. Shelves are obstacles and robots must not move through them. The terrain can be generated in the program and/or read from file(s) for greater variety.
Prompts: How will you represent shelves and aisles on the map? How will you enforce valid movement? How will you ensure robots spawn only at the four corners? How will you create structured shelf layouts?
5. **User Interface:** Your program should have **two modes**: interactive and batch. To run in interactive mode, the program is run with: `python3 warehouse.py -i` Interactive mode should ask for the warehouse dimensions, number of robots, number of goods, simulation length, and any other variables you choose to include. In batch mode, command line parameters should be used to load the warehouse terrain and simulation parameters, for example: `python3 warehouse.py -f map1.csv -p params1.csv`
Prompts: How will you format the parameter file? What variables will be controlled in the command line? How might you use batch mode with an automated script or repeated experiments?
6. **Simulation:** Your program should provide **realtime** and **summary statistics** for each simulation. This should use subplotting so that the warehouse map is shown in one subplot and one or more statistics are shown in other subplot(s). A final summary printed, plotted, or saved to file can provide summary information for the scenario.
Prompts: What interesting statistics could you track and plot? Which statistics are useful for comparing different warehouse scenarios? What summary information best reflects system performance?
7. **Statistics:** Your program should provide realtime and summary statistics of each of your simulations. This should use subplotting to have the park map in one subplot and statistics plotted in one or more subplots. A final summary printed/plotted/saved to file can give summary information for the scenario.
Prompts: What interesting statics could you track and plot? Which statistics are likely to be of interest based on the scenarios?

There are marks allocated for flexibility and usability. For example, using input files, varying warehouse size, varying shelf layouts, changing numbers of robots or goods, or changing simulation parameters can produce very different simulations. You can begin with hard-coded/generated values and filenames, but you should move to prompting for values or, preferably, use command line arguments and/or configuration files to control the simulation.

Your code should include comments to explain what each section does and how. Apply **PEP 8** and other style guides throughout, as this will affect your readability score in the marking. Also beware of using `while True`, `break`, `continue`, and global variables: these are all strongly discouraged in the unit and may significantly reduce your code quality score if used unnecessarily.

Feel free to re-use code and approaches from the lectures and practicals. However, remember to cite or self-cite your sources. If you submit work that you have already submitted for a previous assessment (in this unit or any other) you must specifically state this.

There will be bonus marks for additional functionality and the use of more advanced programming techniques (e.g. interactivity, high quality visualisation, advanced targeting logic, sophisticated warehouse generation, parameter sweep, etc.), but only if they are sensible and done well. Make sure to discuss the additional work in your Report. This will be much easier if you make notes and keep old (incremental) versions of your code. Beyond the working program, you will submit a document: the **Project Report**, worth **40%** of the assignment marks. This is described in Section 3.1.

3. Submission

Submit electronically via Blackboard. You can submit multiple times – we will only mark the last attempt. This can save you from disasters! Take care not to submit your last version late though. Read the submission instructions very carefully.

You should submit a single file, which should be zipped (.zip). Check that you can decompress it successfully. The submission file must be named `FOP_Assignment_<id>` where the `<id>` is replaced by your student id. There should be no spaces in the file name; use underscores as shown.

The file must contain the following:

- **Code** – python code and supporting files, i.e. all files needed to run your program, including input files.
- **README** file including short descriptions of all files and **dependencies**, and information on how to run the program.
- **Report** for your code, as described in Section 3.1.
- **You will also need to submit the Report to TurnItIn.**

Make sure that your zip file contains all items to be marked is required. Anything not included in your zip submission will not be marked. It is your responsibility to make sure that your submission is complete and correct – submitted to the main assignment link as a **single zip file**.

3.1 Project Report

You need to submit your Report in Word **doc or pdf** format. You will need to describe how you approached the implementation of the simulation, and explain to users how to run the program. You will then showcase the application(s) you have developed, and use them to explore the simulation outputs. This exploration would include changing parameters, simulation time and perhaps comparing outcomes if you switch various features on/off.

THE REPORT MUST BE SUBMITTED THROUGH TURNITIN AND IN THE ZIP FILE

Your **Project Report** will be around 10 pages and should include the following:

1. **Overview** (2 marks) describe your program's purpose and implemented features.
2. **User Guide** (2 marks) how to use your simulation (and parameter sweep code, if applicable)
3. **Traceability Matrix** (10 marks) of features, implementation and testing of your code.
The matrix should be a table with columns for:
 - i. **Feature** - numbered for easy referencing
 - ii. **Code reference(s)** – reference to files/classes/methods or snippets of code only, do not put the whole program in the report
 - iii. **Test reference(s)** – test code or describe how you tested your feature was correctly implemented
 - iv. **Test result** - Pass/Fail/Partial Pass/Not Implemented
 - v. **Completion date** - N/A if not implemented
4. **Discussion** (10 marks) of implemented features (referring to the Traceability Matrix), explaining how they work and how you implemented them. A UML Class Diagram should be included for objects and their relationships.
5. **Showcase** (10 marks) of code output, including **three** different scenarios, e.g. different terrain, strategies and or numbers of objects to demonstrate your code:
 - a. **Introduction:** (4 marks) Describe how you have chosen to set up and compare the simulations for the showcase. Include commands, input files – anything needed to reproduce your results.
 - b. **Discussion:** (3x2 marks) Show and discuss each scenario's outputs/results.
6. **Conclusion** (2 marks) reflection on your assignment with respect to the specification.
7. **Future Work** (2 marks) further investigations and/or extensions that could follow.
8. **References** (2 marks)

A report template is available on Blackboard.

3.2 Marking

Marks will be awarded to your submission as follows:

- **[30 marks] Code Features.** Based on your implementation and documentation
- **[30 marks] Demonstration.** Students will demonstrate their code and respond to questions from the markers. Marks are assigned for each feature implemented and for the usability and flexibility of the code.
- **[40 marks] Project Report.** As described in section 3.1.

Marks will be lost for not following specifications outlined in this document, which includes incorrect submission format and content.

3.3 Requirements for passing the unit

Please note: As specified in the unit outline, it is necessary to have a reasonable attempt on the assignment in order to pass the unit. Your assignment must score at least **30% (before penalties)** to be considered a reasonable attempt. We have given you the mark breakdown in Section 3.2. Note that the marks indicated in this section represent maximums, achieved only if you completely satisfy the requirements of the relevant section.

Plagiarism is a serious offence. This assignment has many correct solutions so plagiarism will be easy for us to detect (and we will). For information about plagiarism, please refer to <http://academicintegrity.curtin.edu.au>.

You will be asked to explain parts of your code and the reason for choices that you have made during the demonstration. **A failure to display knowledge required to have produced the code will most likely result in being formally accused of cheating.**

Finally, be sure to secure your code. If someone else gets access to your code (e.g. because you left it on a lab machine, lost a USB drive containing the code or put it on a public repository) you will be held partially responsible for any plagiarism that results.

3.4 Late Submission

As specified in the unit outline, you must submit the assignment on the due date. If there are reasons you cannot submit on time, you should apply formally for an Assessment Extension. If you submit your assignment late (without an extension), you will be penalised based on the number of days it is late.

Students with a **Curtin Access Plan** should include a submission note to indicate the extra time they have taken, ensuring they have submitted the CAP to Blackboard for us to check.

3.5 Clarifications and Amendments

This assignment specification may be clarified and/or amended at any time. Such clarifications and amendments will be announced in the lecture and on the unit's Blackboard page. These clarifications and amendments form part of the assignment specification and may include things that affect mark allocations or specific tasks. It is your responsibility to be aware of these, either by attending the lectures, watching the iLecture and/or monitoring the Blackboard page.